LLM-Based Battery Datasheet Information Extraction and Validation

Rapport de Projet — Données Complexes

Groupe

Mai 2026

Table des matières

1 Introduction **2**
1.1 Jeu de données 2

2 Architecture du Pipeline **2**

3 Extraction LLM (Niveau 1) **3**
3.1 Parsing PDF 3
3.2 Schéma d'extraction 3
3.3 Stratégies de prompting 3
3.3.1 Zero-Shot 3
3.3.2 Few-Shot 4
3.4 Résultats d'extraction 4
3.4.1 Résultats Zero-Shot 4
3.4.2 Résultats Few-Shot 5
3.5 Analyse des résultats 5

4 Graphe de Connaissances (Niveau 2) **5**
4.1 Ontologie 5
4.2 Construction et peuplement du graphe 6

5 Détection d'Hallucinations (Niveau 3) **6**
5.1 Validation par règles Python 6
5.2 Validation par requêtes SPARQL sur le graphe 7
5.3 Résultats de validation 7

6 Discussion **7**
6.1 Apport du graphe de connaissances 7
6.2 Limites et perspectives 8

7 Répartition des Tâches **8**

8 Conclusion **8**

1 Introduction

Les fiches techniques (datasheets) de batteries contiennent des spécifications critiques telles que la tension nominale, la capacité, le taux de décharge, la plage de température de fonctionnement et la résistance interne. Cependant, ces documents sont souvent fournis dans des formats non structurés (PDF), ce qui rend l'intégration automatique dans des bases de données difficile.

Les modèles de langage (LLMs) offrent de nouvelles possibilités pour extraire automatiquement des informations structurées à partir de tels documents techniques. Néanmoins, les LLMs peuvent générer des sorties incohérentes ou des valeurs hallucinées qui ne sont pas présentes dans le document original.

Ce projet vise à concevoir un pipeline intelligent qui **extrait**, **structure** et **valide** les spécifications de batteries en utilisant des LLMs et des graphes de connaissances.

1.1 Jeu de données

Le projet utilise 4 fiches techniques de batteries couvrant différentes chimies et formats :

Table 1 – Batteries analysées

Modèle	**Fabricant**	**Chimie**	**Tension**	**Capacité**
16340	DNK Power	LiFePO4	3.2 V	500 mAh
PL-9059156	BatterySpace	Li-Polymer	3.7 V	10 000 mAh
FFH3DS (86Ah)	CATL	LiFePO4	3.2 V	86 000 mAh
LIR2450	EEMB	Li-ion	3.6 V	120 mAh

2 Architecture du Pipeline

Le système est organisé en un pipeline séquentiel. Le texte brut est d'abord extrait des PDF, puis soumis au LLM qui produit un JSON structuré. Ces résultats sont évalués par rapport à une vérité terrain et alimentent un graphe de connaissances RDF, qui est ensuite interrogé pour détecter d'éventuelles hallucinations.

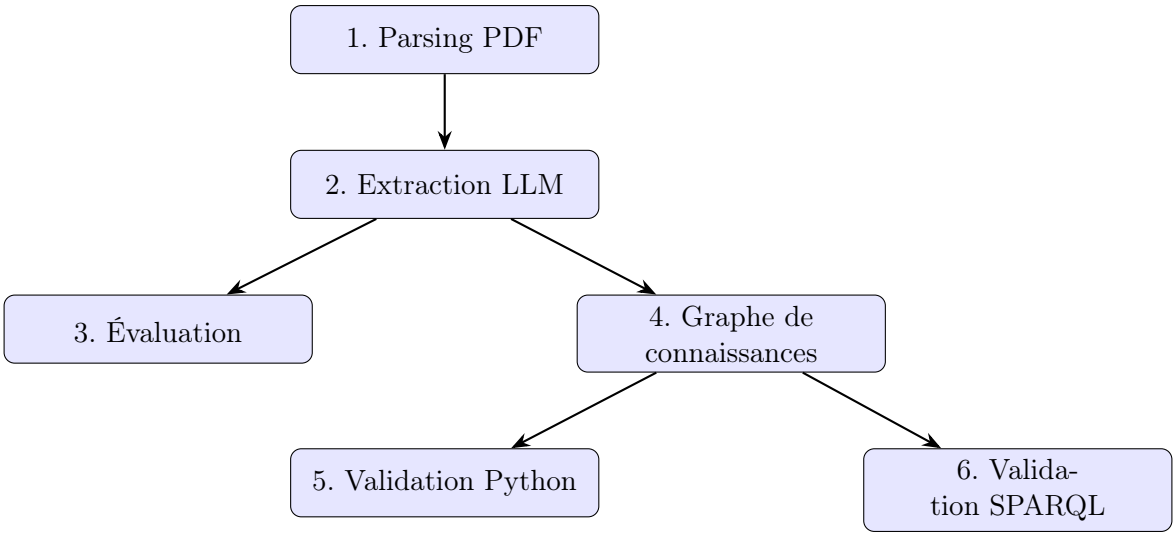

FIGURE 1 – Architecture du pipeline

3 Extraction LLM (Niveau 1)

3.1 Parsing PDF

La première étape consiste à extraire le texte brut de chaque page du PDF à l'aide de la bibliothèque PyMuPDF. Un nettoyage est appliqué pour supprimer les espaces multiples, les lignes vides répétées et les caractères isolés, afin de fournir au LLM un texte aussi propre que possible.

3.2 Schéma d'extraction

Pour garantir que le LLM produise une sortie exploitable, nous définissons un schéma strict de champs à extraire. Ce schéma est implémenté avec **Pydantic**, une bibliothèque Python de validation de données. Pydantic permet de définir des modèles de données typés : chaque champ a un type attendu (float, int, string) et peut être optionnel. Quand le LLM renvoie un JSON, Pydantic vérifie automatiquement que la structure et les types sont corrects, rejetant toute réponse mal formée.

Les champs couvrent :

- **Identité** : modèle, fabricant, chimie
- **Électrique** : tension nominale, capacité, résistance interne, tensions de charge/décharge, courants standard et maximum
- **Thermique** : températures de fonctionnement et de stockage (min/max)
- **Physique et cycle de vie** : poids, énergie, nombre de cycles, taux d'auto-décharge

3.3 Stratégies de prompting

Deux stratégies de prompting sont implémentées et comparées :

3.3.1 Zero-Shot

Le LLM reçoit uniquement les règles d'extraction, la description du schéma JSON attendu et le texte du datasheet. Aucun exemple concret n'est fourni : le modèle doit

inférer le format de sortie à partir de la seule description du schéma.

3.3.2 Few-Shot

En plus des mêmes instructions, un exemple complet est inclus dans la conversation : une fiche technique fictive et le JSON correspondant. Cet exemple montre explicitement comment convertir les C-rates en courants réels (par ex. 0.2C pour 500 mAh = 0.1 A) et comment gérer les valeurs absentes (`null`).

3.4 Résultats d'extraction

Les résultats sont évalués en comparant chaque champ extrait à une vérité terrain annotée manuellement. Les métriques utilisées sont :

- **Precision** : parmi les valeurs que le LLM a extraites, quelle proportion est correcte ?
- **Recall** : parmi les valeurs attendues dans la vérité terrain, quelle proportion a été trouvée ?
- **F1 Score** : moyenne harmonique de la precision et du recall
- **Accuracy** : proportion globale de prédictions correctes, incluant les vrais négatifs

Les valeurs numériques sont comparées avec une tolérance de 5%, et les chaînes de caractères utilisent une correspondance par inclusion.

Comprendre les métriques. Pour chaque champ, la comparaison avec la vérité terrain produit l'un des quatre cas suivants :

- **True Positive (TP)** : le LLM a extrait une valeur et elle est correcte. Exemple : le LLM renvoie `nominal_voltage_V: 3.2` et la vérité terrain confirme 3.2 V.
- **False Positive (FP)** : le LLM a extrait une valeur mais elle est fausse. Exemple : le LLM renvoie `chemistry: "Li-ion"` alors que la vérité terrain indique `"LiFePO4"`.
- **False Negative (FN)** : la vérité terrain contient une valeur mais le LLM ne l'a pas trouvée (il a renvoyé `null`).
- **True Negative (TN)** : la vérité terrain n'a pas de valeur pour ce champ et le LLM a correctement renvoyé `null`.

3.4.1 Résultats Zero-Shot

Table 2 – Résultats d'évaluation — stratégie zero-shot, Gemini 2.5 Flash

Batterie	TP	FP	FN	TN	Prec.	Recall	F1	Acc.
16340	13	1	0	7	92.9%	100%	96.3%	95.2%
9059156	17	0	1	3	100%	94.4%	97.1%	95.2%
FFH3DS	20	0	0	1	100%	100%	100%	100%
LIR2450	17	3	0	1	85.0%	100%	91.9%	85.7%
Agrégé	**67**	**4**	**1**	**12**	**94.4%**	**98.5%**	**96.4%**	**94.0%**

3.4.2 Résultats Few-Shot

Table 3 – Résultats d'évaluation — stratégie few-shot, Gemini 2.5 Flash

Batterie	TP	FP	FN	TN	Prec.	Recall	F1	Acc.
16340	14	0	0	7	100%	100%	100%	100%
9059156	12	6	1	2	66.7%	92.3%	77.4%	66.7%
FFH3DS	20	0	0	1	100%	100%	100%	100%
LIR2450	17	3	0	1	85.0%	100%	91.9%	85.7%
Agrégé	**63**	**9**	**1**	**11**	**87.5%**	**98.4%**	**92.6%**	**88.1%**

3.5 Analyse des résultats

Le zero-shot atteint un F1 agrégé de **96.4%**, supérieur au few-shot (**92.6%**). Ce résultat est contre-intuitif : fournir un exemple au LLM n'améliore pas systématiquement l'extraction, et peut même la dégrader.

Le few-shot introduit des erreurs sur la batterie 9059156, où le modèle hallucine plusieurs valeurs numériques (capacité 11 500 au lieu de 10 000 mAh, courants de 2.3 au lieu de 2.0 A). L'exemple fourni semble **biaiser** le modèle vers des conversions de C-rate inappropriées.

Les deux stratégies inventent également des valeurs de température de stockage et de capacité minimale pour la LIR2450, absentes du datasheet.

En revanche, le few-shot excelle sur deux batteries : le 16340 et le FFH3DS obtiennent tous deux un F1 de 100%, là où le zero-shot commet une erreur de chimie (LiFePO4 classé comme Li-ion) sur le 16340.

En résumé :

- Le **zero-shot** est plus fiable en général, avec seulement 4 FP et 1 FN.
- Le **few-shot** peut exceller sur certaines batteries (F1 parfait) mais introduit davantage d'hallucinations numériques (9 FP).
- Les deux stratégies échouent à extraire le fabricant de la batterie 9059156.

Ces erreurs justifient le recours au graphe de connaissances : seule une vérification croisée entre plusieurs champs permet de détecter ce type d'hallucination.

4 Graphe de Connaissances (Niveau 2)

L'objectif de cette partie est d'organiser les données extraites dans une représentation sémantique interrogeable. Plutôt que de stocker les résultats dans un simple fichier JSON, nous construisons un **graphe de connaissances RDF** qui encode explicitement les relations entre les entités (batteries, spécifications, fabricants, chimies). Cette structuration permet ensuite d'interroger les données avec le langage SPARQL et, surtout, de définir des règles de validation qui exploitent les relations entre entités.

4.1 Ontologie

L'ontologie définit le vocabulaire et la structure du graphe. Elle décrit quelles entités existent dans notre domaine et comment elles sont reliées entre elles.

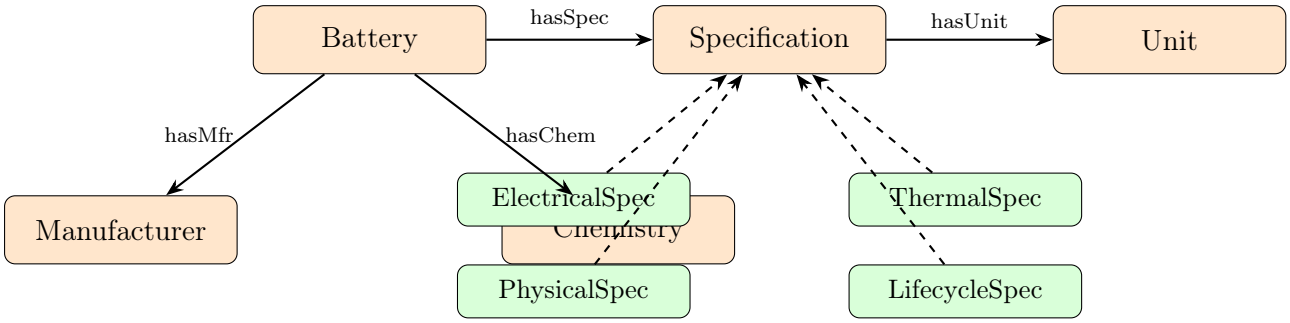

FIGURE 2 – Ontologie OWL — Flèches pleines : relations, pointillés : héritage

Par exemple, une batterie `LIR2450` est reliée à son fabricant `EEMB` via la relation `hasManufacturer`, à sa chimie `Li-ion` via `hasChemistry`, et à chacune de ses spécifications (tension, capacité, etc.) via `hasSpecification`. Chaque spécification porte une valeur numérique et une unité.

4.2 Construction et peuplement du graphe

Le graphe est construit automatiquement à partir des résultats d'extraction JSON. Pour chaque batterie, le système crée les nœuds et relations correspondants dans le graphe RDF. Le graphe résultant contient **431 triplets** (assertions de type sujet-prédicat-objet), représentant **4 batteries** et **57 spécifications**. Il est sérialisé au format Turtle (.ttl), un format standard lisible par les outils du Web Sémantique.

5 Détection d'Hallucinations (Niveau 3)

Les LLMs peuvent produire des valeurs plausibles mais fausses — c'est ce qu'on appelle des **hallucinations**. Un LLM peut par exemple inventer une valeur de capacité qui semble réaliste mais qui n'apparaît nulle part dans le datasheet, ou attribuer une mauvaise chimie à une batterie.

L'objectif de cette partie est de détecter automatiquement ces erreurs **sans accès à la vérité terrain**, en utilisant uniquement les connaissances du domaine. Deux approches complémentaires sont mises en œuvre.

5.1 Validation par règles Python

La première approche consiste à vérifier chaque valeur extraite individuellement, en appliquant des contraintes physiques connues. Par exemple :

- La tension de décharge (cutoff) doit être inférieure à la tension nominale, elle-même inférieure à la tension de charge : autrement, la batterie ne pourrait pas fonctionner.
- Les températures de fonctionnement doivent rester dans des bornes physiquement réalistes.
- Le taux d'auto-décharge doit rester entre 0 et 30%/mois.
- Les valeurs de capacité, résistance et poids doivent être strictement positives.

Chaque anomalie est classée par sévérité : INFO, WARNING, ERROR ou HALLUCINATION.

5.2 Validation par requêtes SPARQL sur le graphe

La validation Python vérifie chaque champ isolément. Mais certaines hallucinations ne sont détectables qu'en **croisant plusieurs champs entre eux**. C'est là qu'intervient le graphe de connaissances : en interrogeant les relations sémantiques du graphe RDF via SPARQL, on peut détecter des incohérences que l'analyse individuelle ne voit pas.

L'exemple le plus parlant est la **cohérence chimie/tension**. Le LLM a extrait la chimie « Li-ion » pour certaines batteries, mais leurs tensions (nominale 3.2 V, charge 3.65 V) sont caractéristiques du LiFePO4. Individuellement, chaque valeur est valide : 3.2 V est une tension acceptable, et « Li-ion » est une chimie réelle. Ce n'est qu'en croisant chimie et profil de tension dans le graphe qu'on révèle l'incohérence.

Listing 1 – Détection d'hallucination chimie/tension via SPARQL

```
SELECT ?model ?chemistry ?nomV ?chargeV WHERE {
    ?b a batt:Battery ; batt:batteryModel ?model .
    ?b batt:hasChemistry ?c .
    ?c batt:chemistryType ?chemistry .
    ?b batt:hasSpecification ?s1 .
    ?s1 batt:specName "Nominal␣Voltage" ;
        batt:hasValue ?nomV .
    ?b batt:hasSpecification ?s2 .
    ?s2 batt:specName "Charge␣Voltage" ;
        batt:hasValue ?chargeV .
    FILTER(?chemistry != "LiFePO4"
        && xsd:float(?nomV) >= 3.0
        && xsd:float(?nomV) <= 3.4
        && xsd:float(?chargeV) >= 3.5
        && xsd:float(?chargeV) <= 3.75)
}
```

D'autres règles SPARQL vérifient l'ordre des tensions, la cohérence énergétique, l'ordre des températures et des courants.

5.3 Résultats de validation

Table 4 – Comparaison des deux méthodes de validation

Méthode	Issues détectées	Type
Validation Python (champ par champ)	0	—
Validation SPARQL (graphe de connaissances)	2	HALLUCINATION

Ce résultat illustre l'apport concret du graphe : la validation Python ne détecte aucune anomalie car chaque valeur est individuellement valide. La validation SPARQL détecte les **2 hallucinations de chimie** en croisant la chimie déclarée avec le profil de tension.

6 Discussion

6.1 Apport du graphe de connaissances

Le résultat principal de ce projet est la démonstration que combiner un LLM avec un graphe de connaissances améliore la fiabilité de l'extraction. Le LLM seul atteint au

mieux un F1 de 97.8% (zero-shot), mais les erreurs de chimie restantes ne sont détectables que par la validation graph-based.

6.2 Limites et perspectives

— **Parsing tabulaire** : Le parsing textuel peut perdre la structure des tableaux. Un outil spécialisé améliorerait l'extraction.

— **Échelle** : Le jeu de données (4 batteries) est limité. Un corpus plus large permettrait de mieux évaluer la robustesse.

— **SHACL** : L'ontologie pourrait être enrichie avec des contraintes SHACL pour une validation encore plus formelle.

7 Répartition des Tâches

Membre	Contributions
Membre 1	Parsing PDF, extraction LLM, prompts zero-shot/few-shot, schéma Pydantic
Membre 2	Ontologie OWL, construction du graphe RDF, requêtes SPARQL
Membre 3	Validation (règles Python + SPARQL), évaluation, ground truth, rapport

8 Conclusion

Ce projet démontre qu'un pipeline combinant LLMs et graphe de connaissances constitue une approche efficace pour l'extraction et la validation de données techniques industrielles. Le système atteint un F1 de 97.8% en zero-shot et 92.6% en few-shot, et surtout, la validation graph-based via SPARQL permet de détecter des hallucinations invisibles aux méthodes classiques.